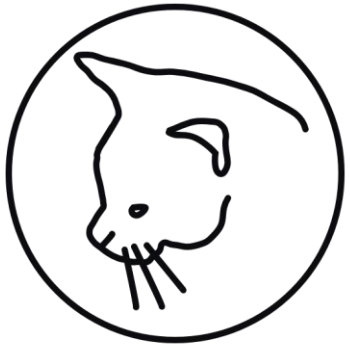


Автоматизоване тестування

Лекція 7

з дисципліни «Якість ПЗ та тестування»

Викладач: Олена Сергіївна Куроп’ятник

Поняття автоматизованого тестування

Автоматизоване тестування передбачає використання інструменту автоматизації для виконання набору тестів.

Автоматизується:

- ✓ введення тестових даних в систему, яку тестують,
- ✓ порівняння очікуваних та фактичних результатів,
- ✓ генерація детального звіту про тестування.

Мета автоматизації : скорочення кількості тестів, які потрібно запускати вручну, а не усунення ручного тестування в цілому.

Доцільність автоматизації тестування

- Ручне тестування через великі обсяги вимагає багато часу та грошей.
- Доволі складно протестувати мультимовні системи вручну.
- Можливість запустити автоматичний тест без нагляду.
- Автоматизація збільшує швидкість виконання тесту.
- Автоматизація допомагає збільшити покриття тестами (Test Coverage).
- Ручне тестування може бути нудним - веде до випадкових помилок.

Мотиви використання автоматизації

Автоматизація процесів тестування буде особливо актуальною, якщо продукт відповідає таким критеріям:

- ✓ сценарії регулярно повторюються;
- ✓ сценарії трудомісткі, складні й не підходять для ручної перевірки;
- ✓ перевірка тестових випадків займає багато часу;
- ✓ сценарії для високонавантажених додатків.

Ручне vs автоматизоване тестування



Типи тестування, які рекомендується автоматизувати

- **Тестування продуктивності** (навантажувальне, стресове) проводиться з метою перевірки працездатності продукту в умовах, максимально наближених до реальних, з очікуваними навантаженнями та обсягом даних.
- **Регресивне тестування** на коректність функціональності застосовується на сервісах, які регулярно змінюються (нові білди, нові версії ПЗ). Завдання РТ — переконатися, що нові зміни, внесені в код, не порушили роботу ПЗ
- **Конфігураційне тестування** застосовується для перевірки працездатності продукту на різних операційних системах і в умовах змін в конфігураціях. При розробці мобільних застосункців КТ дозволяє контролювати роботу продукту на різних мобільних пристроях з урахуванням розмірів і роздільної здатності екрану, операційних систем, їх версій і т.п.
- **Тестування локалізації** відповідає за налаштування продукту, з урахуванням адаптації його для країни покупця — переклад на обрану мову елементів інтерфейсу, контроль часового пояса і формату дати, переклад текстів, розкладка клавіатури та ін. Сюди ж відносяться і багатомовні сайти, які дуже складно тестувати в ручному режимі.
- **Інтеграційне тестування** застосовується для групових тестів, які об’єднують програмні модулі, створені декількома програмістами.

Види автоматизованого тестування

Автоматизація тестування коду (Code-driven testing)

- тестування на рівні програмних модулів, класів та бібліотек (фактично, автоматичні юніт-тести);

Автоматизація тестування графічного інтерфейсу користувача (Graphical user interface testing)

- спеціальна програма (фреймворк автоматизації тестування) дозволяє генерувати користувальницькі дії – натиснення кнопок, кліки мишкою, та відслідковувати реакцію програми на ці дії – чи відповідає вона специфікації.

Автоматизація тестування API (Application Programming Interface) – програмного інтерфейсу програми.

- Тестуються інтерфейси, призначені для взаємодії, наприклад, з іншими програмами або з користувачем. Тут, як правило, використовуються спеціальні фреймворки.

Поняття модульного тестування

Модульне тестування – процес тестування окремих блоків, підпрограм, класів або процедур, що утворюють велику програму.

Метою є порівняння функцій, які реалізуються модулем, зі специфікаціями, що описують його функціональні або інтерфейсні характеристики.

Модульне тестування сприяє:

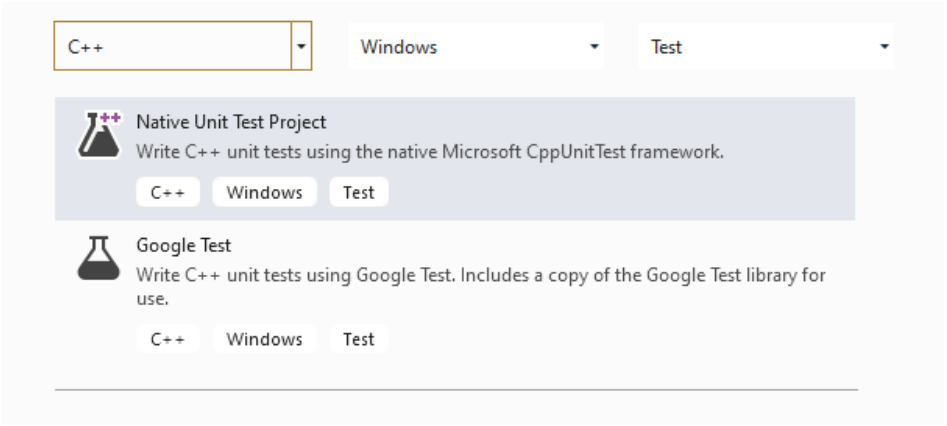
- підвищенню ефективності тестування, оскільки увага зосереджена на меншій частині;
- полегшенню налагодження програм;
- паралельності процесів тестування.

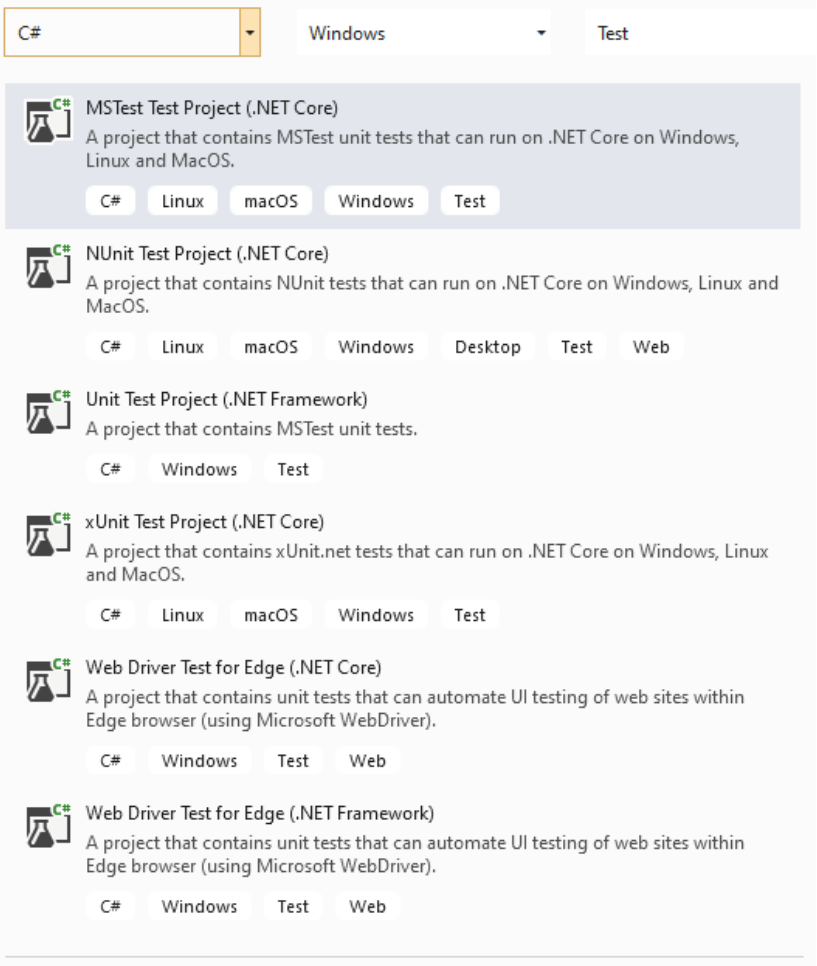
Проектування тестів

- Використання специфікації та тексту програми.
- Використання методів білої скриньки для створення тестів.
- Доповнення тестів на основі методів чорної скриньки.

Автоматизація модульного тестування

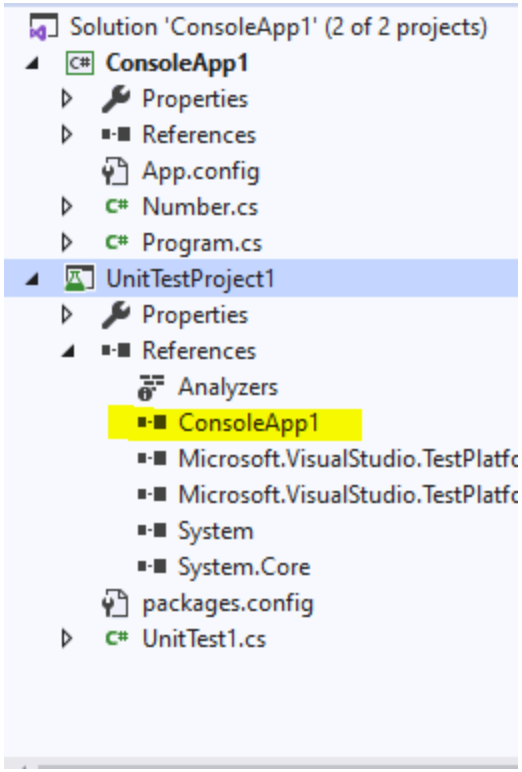
- Засоби MS Visual Studio





Порядок unit-тестування C#

- Написати модулі, які треба протестувати. Помітити відповідні класи public
- Створити проект unit-тесту
- До проекту unit-тесту додати посилання на проект, який тестується
- Підключити в проекті тесту проект, який буде тестуватися (наприклад, using ConsoleApp1;)
- Прописати тестовий метод
- Запустити тести на виконання



Вибір шаблону тесту (частина 1)

Назва шаблону	Вид тестування	Для чого потрібен і коли обирати
xUnit Test Project	Modul / Unit, Integration	Сучасний стандарт. Обирайте для більшості нових .NET-проєктів. Забезпечує високу ізоляцію тестів та чистоту коду без застарілих атрибутів.
NUnit 3 Test Project	Modul / Unit, Integration	Гнучке тестування. Чудово підходить для складних параметризованих тестів завдяки багатому синтаксису fluent-тверджень (Assert.That).

Вибір шаблону тесту (частина 2)

Назва шаблону	Вид тестування	Для чого потрібен і коли обирати
MSTest Test Project	Modul / Unit, Integration	Екосистема Microsoft. Обирайте для нативної інтеграції з Azure DevOps та tooling-ом Microsoft без додаткових сторонніх залежностей.
NUnit Playwright Test Project	E2E / Web UI	Автоматизація UI / Веб. Для тестування реальних браузерів (Chromium, Firefox, WebKit), якщо ваша команда використовує синтаксис та підготовлений стан, дані або ресурс NUnit.

Вибір шаблону тесту (частина 3)

Назва шаблону	Вид тестування	Для чого потрібен і коли обирати
MSTest Playwright Test Project	E2E / Web UI	Автоматизація UI / Веб. Для тестування сценаріїв користувача у браузері з використанням ранера та атрибутів MSTest.
Unit Test Project (.NET Framework)	Modul / Unit	Legacy-проєкти. Потрібен виключно для підтримки та тестування старих проєктів, прив'язаних до .NET Framework і Windows.

Клас Assert

Допоміжний клас для перевірки різних умов у модульних тестах. Якщо умова, що перевіряється, не виконується, створюється виняток.

Основні методи:

- AreEqual/AreNotEqual
- AreNotSame/AreSame
- IsFalse/IsTrue
- IsInstanceOfType/IsNotInstanceOfType
- IsNotNull

Клас CollectionAssert

Колекція допоміжних класів для тестування різних умов, пов’язаних із колекціями в модульних тестах. Якщо умова, що перевіряється, не виконується, створюється виняток.

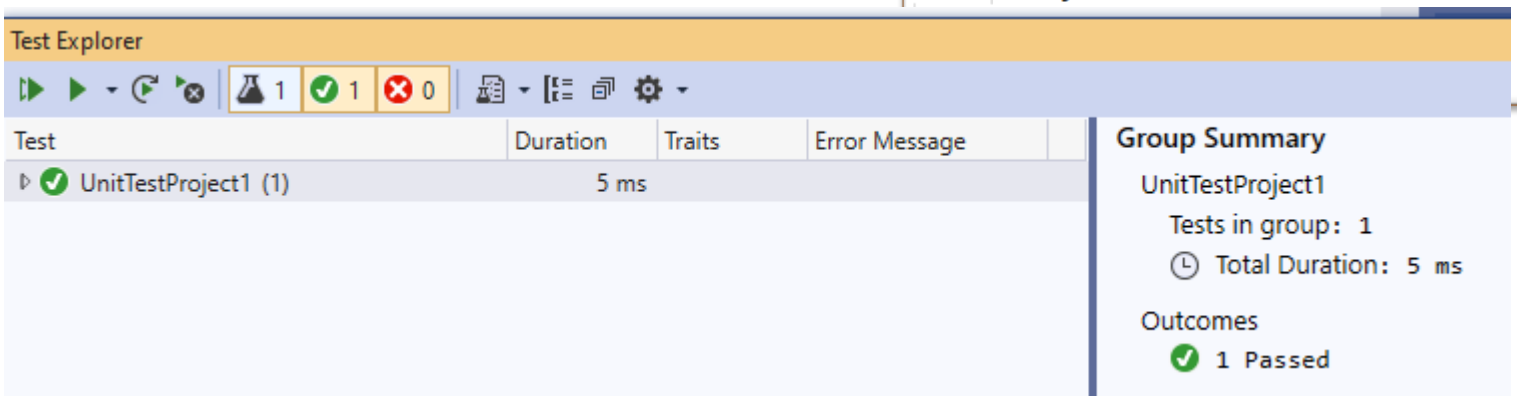
Основні методи:

- AllItemsAreInstancesOfType
- AllItemsAreNotNull
- AllItemsAreUnique
- AreEqual/AreNotEqual
- AreEquivalent/AreNotEquivalent
- Contains/DoesNotContain
- IsSubsetOf/IsNotSubsetOf

Приклад тестування

```
namespace ConsoleApp1
{
    2 references
    public class Number
    {
        1 reference
        public bool IsPositive(int value)
        {
            return value > 0;
        }
    }
}
```

```
using ConsoleApp1;
namespace UnitTestProject1
{
    [TestClass]
    0 references
    public class UnitTest1
    {
        [TestMethod]
        0 references
        public void TestMethod1()
        {
            Number obj=new Number();
            int value = 10;
            Assert.IsTrue(obj.IsPositive(value));
        }
```



Резюме

Автоматизоване тестування передбачає використання спеціального програмного забезпечення - зовнішніх інструментів

Автоматизоване тестування не виключає ручне

Unit-тестування – різновид автоматизованого тестування

Для unit-тестування можуть використовуватися вбудовані засоби IDE

Дякую за увагу!